

本周规划

2+2

qjl, clc-全栈整合+AI 引擎驱动

- (CLC&QJL)直接学习使用 **LangChain** 或 **LlamaIndex** 框架。调用现成的 PDF Loader，用现成的 TextSplitter 切块，调 OpenAI/通义千问/智谱等成熟 API 生成 Embedding 并存入现成的本地向量库（Chroma 或 FAISS）
- **QJL (API 封装与前端)**：把跑通的 Python 脚本包进 Flask 里变成接口（如 `/api/ai/chat`）；同时用 Vue + Element-Plus 快速拼凑一个聊天对话框和上传按钮。

hmj, lzy-Java 业务支撑

- **HMJ (Java 业务主导)**：放弃传统的 SSM 配置，直接上手 **Spring Boot + MyBatis-Plus**（能省去大量繁琐的 XML 配置，对新手极度友好）。负责搭建项目脚手架，设计简单的数据库表。
- **LZY (后端跟学与开发)**：跟着 HMJ 配置环境。HMJ 负责写难度稍微大一点的“用户注册/登录 (JWT)”，LZY 负责写简单的“单表 CRUD（比如历史记录表）”和学习“如何接收前端传来的文件并保存到本地”。里增加一条记录（比如：张三，上传了xxx.pdf，时间xx:xx）

第一周

- QJL

- **Flask 接口设计**：只需要两个。
 - `POST /api/upload`：接收前端传来的 PDF 文件，把它丢给跑通的 Python 脚本进行“解析->切块->存入 Chroma”。成功后返回 `{ "status": "success", "msg": "知识库构建完成" }`。
 - `POST /api/chat`：接收前端的 JSON `{ "question": "请假流程是什么?" }`，调用跑通的 Python 脚本去 Chroma 搜结果，喂给大模型，拿到字符串后返回 `{ "answer": "请假流程是..." }`。
- **Vue 前端**：不需要花里胡哨，找一个 Element-Plus 的 `el-upload` 组件做上传，右边用 `v-for` 渲染一个对话数组 `[{role: 'user', content: '...'},{role: 'ai', content: '...'}]` 即可。

- CLC+QJL

用 Python 写一个 `rag_core.py`，核心只需要 3 个现成的类：

- **PDF 解析选型**：在 LangChain 中，不要用最基础的 `PyPDFLoader`（有时中文换行处理不好），建议安装 `PyMuPDF`，使用 `PyMuPDFLoader`，它对中文字符提取和基础排版保留效果更好。
- **切块参数 (Chunking)**：使用 `RecursiveCharacterTextSplitter`。参数直接抄作业：`chunk_size=500`（每块大概500字），`chunk_overlap=50`（块与块重叠50字，防止一句话被从中间截断）。
- **向量数据库**：直接用 **Chroma**。它是轻量级本地库，不需要安装额外的服务（像 MySQL 那样），只要 `pip install chromadb`，数据会直接存在项目的一个本地文件夹里，超级适合 MVP。

- HMJ + LZY 的 Java 后端（数据库表怎么建？

LZY 是新手，所以 Java 业务系统要足够克制，用MySQL建两个表就够了：

- 表1: `user` (用户表)
 - `id` (主键), `username` (用户名), `password` (密码, MVP阶段 明文 存也没事, 别搞加密把新手整晕了).
- 表2: `chat_history` (聊天记录表 - 让 LZY 练手 CRUD)
 - `id` (主键)
 - `user_id` (关联用户)
 - `file_name` (比如 "员工手册.pdf", 记录这次对话基于什么文件)
 - `question` (用户问了啥, TEXT类型)
 - `answer` (AI回了啥, TEXT类型)
 - `create_time` (提问时间)

DDL

🚩 DDL 1: 环境与基建 (周四晚 22:00)

目标: 所有人跑通“Hello World”, 把最恶心的环境问题全部解决。

- [CLC + QJL] AI 组:
 - 装好 Python 环境, 搞定大模型 API Key。
 - 写一段不到 10 行的代码: 用 `PyMuPDFLoader` 成功读取一篇纯文本 PDF, 并在终端 `print()` 打印出文字, 证明解析库没问题。
- [QJL] 前端组:
 - 初始化 Vue3 + Element-Plus 项目。
 - 画出静态页面: 左边有个上传按钮, 右边有个能滚动的聊天框 (写死两条假对话即可)。
- [HMJ] + LZY] Java 组:
 - LZY 在本地装好 JDK、IDEA、MySQL, HMJ 帮 LZY 建好 `user` 和 `chat_history` 两张表。
 - 跑通 Spring Boot 项目, 写一个 `@GetMapping("/test")` 返回 "Hello World", 在浏览器能看到。

🚩 DDL 2: 核心业务单点跑通 (周六晚 22:00)

目标: 各个模块能在不依赖前端的情况下, 自己用黑框或测试工具跑通。

- [CLC + QJL] AI 组 (最核心任务):
 - 完成 `rag_core.py` 脚本。
 - 验收标准: 在 Python 终端里运行, 输入一句问题 (如“请假怎么请”), 脚本能通过 Chroma 检索 PDF 片段, 并调用大模型打印出正确的回答。
- [HMJ] Java 核心业务:
 - 完成 `user` 表的注册和登录接口。
 - 验收标准: 用 ApiPost / Postman 发送账号密码, 能返回成功提示 (或 Token)。
- [LZY] Java 跟学业务:
 - 完成 `chat_history` 表的增、查接口 (不需要删和改)。
 - 验收标准: 用 ApiPost 传一段假的问题和答案, 能成功存入 MySQL 数据库, 并在 Navicat/DataGrip 里

看到。

▶ DDL 3: API 化与第一周大整合 (下周二晚 22:00)

目标：所有代码变成 API 接口，前端能发通请求（哪怕有 Bug，但链路必须通）。

- **[QJL] API 封装：**
 - 把 `rag_core.py` 套上 Flask，变成 `POST /api/upload` 和 `POST /api/chat`。
 - 把 Vue 页面与 Flask 接口连通。
 - **验收标准：**在 Vue 页面点按钮上传 PDF，在输入框发问题，页面上能展示 AI 的真实回答（AI 链路全线贯通）。
- **[LZY + HMJ] Java 文件业务：**
 - 实现 `/api/file/upload` 接口（可以请教 QJL 前后端传文件的数据格式）。
 - **验收标准：**用 ApiPost 上传一个 PDF，Java 后端能把它存到电脑的某个文件夹（比如 `D:/uploads/`），并在数据库记录。